DAT32-1

# Intro to Machine Learning

## The ML project cycle through linear regression

Learn machine learning by building one model completely.

Albert School — 16 hours

# Contents


1 The ML taxonomy and the project workflow ........ 2
2 Core vocabulary through regression ........ 2
3 Simple linear regression from first principles ........ 3
4 Gradient descent ........ 3
5 Underfitting and overfitting ........ 4
6 Baseline models ........ 4
7 Evaluation metrics for regression ........ 4
8 Regularization ........ 5
9 Cross-validation ........ 5
10 Hyperparameters ........ 5
11 From simple to multiple regression ........ 6
12 Scikit-learn pipelines ........ 6


## 1 The ML taxonomy and the project workflow

Machine learning builds a model that maps inputs to outputs by fitting it to data rather than by being programmed with explicit rules.

> **Definition 1** **Supervised learning** fits a model on data where each example carries a known target. **Unsupervised learning** works on data with no target, and instead finds structure (groups, directions) in the inputs alone.

> **Definition 2** Within supervised learning, **regression** predicts a continuous numeric target, and **classification** predicts a discrete category. **Clustering** is an unsupervised task that partitions examples into groups.

Given a business problem, the task type is determined by the target: a numeric quantity to predict is regression, a category to assign is classification, and the absence of any target with a request to find groups is clustering.

The project workflow is the same regardless of the model: define the target, assemble and split the data, choose a baseline, train a model, evaluate it on data it has not seen, diagnose underfitting and overfitting, and tune. The rest of this course follows this workflow on a single model, linear regression.

## 2 Core vocabulary through regression

> **Definition 3** A **feature** is an input variable used to predict the target. A **label** is the known value of the target for a given example. A dataset for supervised learning is a table of examples, each with its features and its label.

> **Definition 4** The **training set** is the data used to fit the model. The **validation set** is held out from training and used to compare models and tune choices. The **test set** (also called the **holdout set**) is data used once, at the end, to estimate performance on unseen data. An example used to tune the model must not also be used to report its final performance.

We write a training example as a feature vector $x = (x_1, ..., x_p)$ with label $y$, and the dataset as $n$ such pairs $(x^{(i)}, y^{(i)})$ for $i = 1, ..., n$.

# 3 Simple linear regression from first principles

**Definition 5** **Simple linear regression** models the target as a linear function of one feature:

$$\hat{y} = \beta_0 + \beta_1 x,$$

where $\beta_0$ is the intercept and $\beta_1$ is the slope. The prediction $\hat{y}$ is the model's estimate of the label $y$.

The coefficients are chosen to minimise the discrepancy between predictions and labels, measured by a cost function.

**Definition 6** The **residual sum of squares** (RSS) is the sum of squared errors over the training set:

$$\mathrm{RSS}(\beta_0, \beta_1) = \sum_{i=1}^{n} \left(y^{(i)} - \beta_0 - \beta_1 x^{(i)}\right)^2.$$

**Definition 7** **Ordinary least squares** (OLS) chooses the coefficients that minimise the RSS. For simple regression the minimiser is available analytically:

$$\beta_1 = \frac{\sum_{i=1}^{n} \left(x^{(i)} - \bar{x}\right)\left(y^{(i)} - \bar{y}\right)}{\sum_{i=1}^{n} \left(x^{(i)} - \bar{x}\right)^2}, \quad \beta_0 = \bar{y} - \beta_1 \bar{x},$$

where $\bar{x}$ and $\bar{y}$ are the means of the feature and the label.

**Coefficient interpretation.** The slope $\beta_1$ is the change in the predicted target for a one-unit increase in the feature; the intercept $\beta_0$ is the predicted target when the feature is zero.

**Definition 8** Valid statistical inference on the coefficients requires assumptions: the relationship is linear in the coefficients, the errors have zero mean and constant variance, the errors are uncorrelated, and they are independent of the feature. When these hold, OLS estimates are unbiased.

# 4 Gradient descent

The analytical solution exists for OLS, but most models are fit by iteratively minimising the loss. Gradient descent is that procedure.

**Definition 9** **Gradient descent** minimises a loss $L(\beta)$ by repeatedly stepping in the direction of steepest decrease, i.e. against the gradient:

$$\beta \leftarrow \beta - \alpha \nabla L(\beta),$$

where $\alpha > 0$ is the **learning rate**. Steps repeat until the loss stops decreasing meaningfully (convergence).

For the mean squared error loss in simple regression, the partial derivatives are

$$\frac{\partial L}{\partial \beta_0} = -\frac{2}{n}\sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)}), \quad \frac{\partial L}{\partial \beta_1} = -\frac{2}{n}\sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})x^{(i)}.$$

**Definition 10** The **learning rate** $\alpha$ controls the step size. Too small and convergence is slow; too large and the iterates overshoot the minimum and the loss diverges instead of decreasing.

**Common pitfall** Diverging loss (values growing each iteration) is the signature of a learning rate that is too large, not of a bug in the gradient. Reduce $\alpha$ before suspecting the derivative.

## 5 Underfitting and overfitting

**Definition 11** A model **underfits** when it is too simple to capture the structure in the data: it has high error on both the training set and unseen data. A model **overfits** when it is so flexible that it fits the noise in the training set: it has low training error but high error on unseen data.

In regression, model flexibility is controlled by the **polynomial degree**: fitting $\hat{y} = \beta_0 + \beta_1 x + \ldots + \beta_d x^d$ with a large degree $d$ produces a flexible model that can bend to fit noise, and is therefore sensitive to small changes in the data.

**Definition 12** A **learning curve** plots training error and validation error against the amount of training data or model complexity. Underfitting shows as high error on both curves; overfitting shows as a large gap, with low training error and high validation error.

## 6 Baseline models

**Definition 13** A **baseline** is the simplest defensible predictor, used as the floor a model must beat. For regression the standard baseline is the **mean predictor**, which ignores the features and predicts the mean of the training labels for every example.

A model that does not beat the baseline has no value: it has not extracted any predictive signal from the features beyond what predicting the average already achieves.

## 7 Evaluation metrics for regression

**Definition 14** For predictions $\hat{y}^{(i)}$ against labels $y^{(i)}$ over $n$ examples:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}\left|y^{(i)} - \hat{y}^{(i)}\right|, \quad \text{MSE} = \frac{1}{n}\sum_{i=1}^{n}\left(y^{(i)} - \hat{y}^{(i)}\right)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}.$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}\left(y^{(i)} - \hat{y}^{(i)}\right)^2}{\sum_{i=1}^{n}\left(y^{(i)} - \bar{y}\right)^2}.$$

MAE is in the units of the target and weights all errors equally. MSE and RMSE penalise large errors more heavily because the error is squared; RMSE is back in the units of the target. $R^2$ is the fraction of the target's variance the model explains: $R^2 = 0$ matches the mean predictor, $R^2 = 1$ is perfect.

The choice is justified by the business context: use MAE when all errors matter proportionally to their size, and MSE or RMSE when large errors are disproportionately costly.

# 8 Regularization

**Definition 15** **Regularization** adds a penalty on the size of the coefficients to the loss, to discourage overfitting. **L2 regularization** (ridge) penalises the sum of squared coefficients; **L1 regularization** (lasso) penalises the sum of absolute coefficients:

$$L_{\text{ridge}} = \text{RSS} + \lambda \sum_{j=1}^{p} \beta_j^2, \quad L_{\text{lasso}} = \text{RSS} + \lambda \sum_{j=1}^{p} |\beta_j|.$$

The strength $\lambda \geq 0$ controls how heavily large coefficients are penalised.

L2 shrinks coefficients toward zero without setting them exactly to zero. L1 can drive coefficients exactly to zero, performing feature selection. Prefer L1 when a sparse model using few features is wanted; prefer L2 when all features are expected to contribute and only their magnitudes should be controlled.

# 9 Cross-validation

**Definition 16** A single train/test split gives an unreliable performance estimate because the result depends on which examples happened to fall in the test set.

**Definition 17** **k-fold cross-validation** splits the data into $k$ equal folds. For each fold, the model is trained on the other $k - 1$ folds and evaluated on the held-out fold. The $k$ scores are averaged. Every example is used for evaluation exactly once, so the estimate uses all the data and is more reliable than a single split.

# 10 Hyperparameters

**Definition 18** A **hyperparameter** is a setting fixed before training that is not learned from the data — for example the regularization strength $\lambda$, the polynomial degree, or the learning rate. Parameters (the coefficients) are learned; hyperparameters are chosen.

Hyperparameters are tuned by training with each candidate value and comparing performance on the validation set (or by cross-validation), then selecting the best. Tuning on the test set invalidates it: the test set would then have influenced the model, so it could no longer give an honest estimate of performance on unseen data.

**Common pitfall** The test set is touched once, after all tuning is finished. Selecting a hyperparameter by its test-set score leaks the test set into the model and inflates the reported performance.

## 11 From simple to multiple regression

**Definition 19** **Multiple linear regression** uses several features:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p.$$

Each coefficient $\beta_j$ is the change in the predicted target for a one-unit increase in feature $x_j$ with all other features held fixed.

**Definition 20** **Multicollinearity** is strong correlation among features. When features are collinear, the individual coefficients become unstable and hard to interpret — small changes in the data produce large changes in the coefficients — because the data cannot separate the effect of one correlated feature from another.

Multicollinearity is diagnosed by examining correlations among the features and by observing coefficients that are large, unstable, or have implausible signs.

## 12 Scikit-learn pipelines

**Definition 21** A **scikit-learn pipeline** chains preprocessing steps and a model into a single object. Calling fit on the pipeline applies each step's fit in order; calling predict applies each transformation and then the model. This keeps the workflow reproducible and prevents information from the validation or test data leaking into preprocessing fitted on the training data.

A pipeline is the reproducible form of the whole workflow: the same chained steps are applied identically during cross-validation, hyperparameter tuning, and final evaluation.

**Exercises**

**1.** Given three business problems, identify each as regression, classification, or clustering, and state whether it is supervised or unsupervised.
**2.** For a labelled dataset, identify the features and the label, then split it into training, validation, and test sets and explain the role of each.
**3.** Implement OLS simple linear regression from first principles using the analytical formulas. Interpret the slope and intercept, and state the assumptions required for valid inference.
**4.** Implement gradient descent to fit the same regression. Vary the learning rate and describe convergence, including a rate large enough to diverge.
**5.** Fit polynomials of increasing degree to a noisy dataset. Identify the underfitting and overfitting regimes from the training and validation learning curves.

**6.** Implement the mean-predictor baseline and compare a fitted model against it. Explain why a model that does not beat the baseline has no value.
**7.** Compute MAE, MSE, RMSE, and $R^2$ for a model. Select and justify one metric given a stated business context.
**8.** Apply L1 and L2 regularization to the regression. Describe what each penalises and give a situation where each is preferable.
**9.** Implement k-fold cross-validation and compare its performance estimate to that of a single train/test split.
**10.** Tune the regularization strength on the validation set. Explain why tuning it on the test set would be invalid.
**11.** Extend the model to multiple features. Interpret the coefficients, then introduce two correlated features and diagnose the resulting multicollinearity.
**12.** Build a scikit-learn pipeline chaining preprocessing and the regression model, and run it under cross-validation.